Unique Value Prop

CRUSHED — AI Creative Platform

Your Creative Director, Powered by AI

The Problem

Most AI creative tools expect users to:

- Learn prompt engineering
- Connect complex nodes
- Manually tune models
- Translate briefs into technical instructions

Marketers and creators don’t think in prompts. They think in ideas, goals, and emotions.

The Solution

A conversational AI platform that turns natural language briefs into production-ready creative outputs.

You describe *what you want to achieve*.
CRUSHED handles *how to generate it*.

What Makes It Unique

- **Chat-First Creation**
  Talk naturally. The system asks the right questions—like a senior creative director.
- **High-Fidelity System Prompts**
  Automatically generates structured, multimodal system prompts behind the scenes.
- **Multimodal by Default**
  One brief → Image, Video, and Copy outputs.
- **Built for Commercial Use**
  Trained on ad samples, moodboards, scripts, and treatments—not generic internet noise.

The Value

- No prompt engineering
- No node-based workflows
- Faster ideation → execution
- Consistent, brand-ready outputs

Who It’s For

Marketers · Graphic Designers · Freelancers · Video Producers · Storyboard Artists · Agencies

---

One-Line UVP

“If you can explain the idea, we can produce the creative.”

Key User Personas

Key User Personas for CRUSHED

1. The Performance Marketer

(Growth, Paid Media, Brand Marketing teams)

How they think

- In **objectives**: CTR, ROAS, retention, recall
- In **formats**: ads, thumbnails, hooks, variants
- In **speed & iteration**, not aesthetics alone

Pain with existing tools

- Prompting feels wasteful and inconsistent
- Visual tools don't understand *campaign goals*
- Needs 10–20 variations, not 1 "perfect" image

What they ask (naturally)

- "I need 5 Reel creatives that stop scroll in 2 seconds"
- "This is for a festive sale, performance-first"
- "Make it feel premium but clickable"

Why CRUSHED wins

- Chat interprets **marketing intent**, not just visuals
- Automatically generates:
    - Ad-grade visuals
    - Hook-first copy
    - Format-optimized variants
- No need to learn visual prompting language

Core Value: *Turns business goals into creative outputs.*

---

2. The Graphic Designer

(Brand designers, social designers, freelancers)

How they think

- In **visual references**, not words
- In **mood, hierarchy, balance, style**
- Already creative—but hates technical friction

Pain with existing tools

- Node systems break creative flow

- Prompts feel imprecise and fragile
- Output often “looks AI” and needs heavy fixing

What they ask (naturally)

- “Make this more editorial, like a magazine cover”
- “This should feel minimal, brutalist, luxury”
- “I want this to look designed, not generated”

Why CRUSHED wins

- Conversational Q&A extracts:
    - Visual language
    - Aesthetic intent
    - Design constraints
- System generates **high-fidelity system prompts**, not raw guesses
- Output feels closer to *designed assets*

Core Value: *AI that speaks visual language fluently.*

---

3. The Video Producer / Filmmaker

(Reel editors, DVC teams, indie filmmakers, studios)

How they think

- In **shots, sequences, pacing**
- In **emotion over time**
- In **storyboards, not single frames**

Pain with existing tools

- Video tools are fragmented and technical
- Storyboarding is manual and time-consuming
- AI videos lack narrative continuity

What they ask (naturally)

- “Give me a 5-shot reel structure”
- “This should feel cinematic, slow build, emotional”
- “I want a before–after transformation story”

Why CRUSHED wins

- Chat behaves like a **director’s assistant**
- Generates:
    - Shot lists
    - Visual references

    - Scene-to-scene continuity
- Multimodal flow: text → image → video

Core Value: *From story intent to visual execution.*

---

4. The Freelancer / Solo Creator

(Designers, editors, marketers working alone)

How they think

- In **client briefs**
- In **deliverables & timelines**
- Needs speed + consistency + confidence

Pain with existing tools

- Too many tools to stitch together
- Hard to explain AI outputs to clients
- Inconsistent quality kills trust

What they ask (naturally)

- “This is for a real estate client”
- “Client wants premium but not flashy”
- “Give me something I can show directly”

Why CRUSHED wins

- One chat → multiple client-ready outputs
- Less “AI noise”, more usable work
- Reduces creative anxiety and revision loops

Core Value: *A silent creative partner that upgrades output quality.*

---

5. The Storyboarding Artist / Creative Strategist

(Ad agencies, concept artists, pitch teams)

How they think

- In **ideas before execution**
- In **narrative arcs and treatments**
- In *selling the idea*, not just visuals

Pain with existing tools

- AI tools jump straight to visuals
- No help in structuring the idea

- Moodboards and scripts live separately

What they ask (naturally)

- “Help me visualize this idea”
- “What does this concept look like in frames?”
- “I need this to land emotionally”

Why CRUSHED wins

- Q&A helps **shape the idea**, not just render it
- Generates:
    - Moodboards
    - Story frames
    - Visual narratives
- Perfect for pitches and early-stage ideation

Core Value: *AI that understands creative thinking, not just output generation.*

---

6. The Casual Creator (Secondary Persona)

(Small brands, founders, non-designers)

How they think

- In plain language
- Wants “something that looks good”
- Doesn’t care about models or styles—just results

Pain with existing tools

- Intimidated by AI interfaces
- Doesn’t know “what to ask”
- Gets overwhelmed easily

Why CRUSHED still works (but isn’t core)

- Natural language first
- Guided questioning
- Sensible defaults

Core Value: *Lowering the entry barrier to high-quality creative.*

Persona Hierarchy (Strategic Insight)

Primary Power Users (Your Moat):

1. Performance Marketers

2. Graphic Designers
3. Video Producers

High-Leverage Multipliers:
4. Freelancers
5. Storyboarding Artists

Long-Term Expansion:
6. Casual Creators

This hierarchy matters for:

- Feature prioritization
- Onboarding flows
- Pricing tiers
- GTM messaging

One Critical Takeaway

CRUSHED is **not** a “creator tool”.

It’s a **creative intent translation engine**.

That’s why these personas work—they all:

- Think clearly about *what they want*
- Hate translating that into AI instructions

AI Moat Strategy

CRUSHED — AI Moat Strategy

Short Answer (Executive Summary)

CRUSHED's moat is *not* proprietary foundation models.

Your moat is a **layered AI orchestration system** built on:

- Fine-tuned open-source models (LoRAs / adapters)
- Selective third-party APIs (best-of-breed)
- Proprietary conversational + creative-intent intelligence

You own the "thinking layer," not the raw intelligence.

The 3-Layer AI Moat Architecture

Layer 1: Foundation Models (Commodity Layer)

Strategy: *Do NOT compete here.*

Use:

- Third-party APIs (closed + frontier)
- Open-source base models (for control & cost)

Why:

- Foundation models are capital-intensive
- Differentiation here decays fast
- Users don't care *which model*—they care about outputs

Decision:
➡️ **Leverage, don't build**

Layer 2: Model Adaptation (Fine-Tuning & LoRAs)

Strategy: *Selective ownership*

This is where CRUSHED starts differentiating technically.

You should:

- Fine-tune **open-source models** using:
  - Ad creatives
  - Storyboards
  - Treatments
  - Moodboards
  - Script structures
- Use **LoRAs / adapters** per:
  - Industry
  - Format (Reels, thumbnails, DVCs)

    - Aesthetic cluster (premium, UGC, cinematic, etc.)

Why this matters:

- Improves *creative taste alignment*
- Reduces “AI-generic” outputs
- Allows brand / agency-specific tuning later

Decision:
➡️ **Yes to LoRAs and fine-tuned OSS models**
➡️ **No to training from scratch**

Layer 3: The Real Moat — Creative Intelligence System

This is your **true, defensible moat**.

CRUSHED owns:

- Conversational brief extraction
- Creative questioning logic
- Intent classification
- Prompt synthesis logic
- Multimodal orchestration
- Fidelity scoring + iteration loops

This layer is:

- Proprietary
- Accumulative
- Extremely hard to replicate

Decision:
➡️ **100% build and own**

The Big Decision Matrix

Option	Should CRUSHED Use It?	Why
Proprietary foundation models	❌ No	Capital heavy, weak differentiation
Third-party APIs	✅ Yes	Best quality, fastest iteration
Open-source base models	✅ Yes	Control, cost, customization
Fine-tuned LoRAs	✅ Strong Yes	Creative alignment moat
Prompt libraries	✅ Yes	Compound IP
Node-based pipelines	❌ No	Against UVP
Hard-coded workflows	❌ No	Limits creativity

What the Moat Actually Is (In One Sentence)

CRUSHED's moat is its ability to understand creative intent through conversation and translate it into high-fidelity, multimodal system prompts that consistently produce commercial-grade outputs.

Why This Moat Grows Over Time

Every interaction improves:

- Creative intent detection
- Question sequencing
- Prompt structuring
- Output ranking
- Persona-specific preferences

Over time you accumulate:

- A proprietary **Creative Intent Graph**
- A **Prompt-to-Outcome knowledge base**
- Format-specific creative intelligence

This compounds quietly—and competitors can't scrape it.

Persona-to-Moat Alignment

Performance Marketers

- Moat = campaign-aware prompt synthesis

Designers

- Moat = visual language interpretation

Video Producers

- Moat = narrative continuity + shot logic

Freelancers

- Moat = client-brief normalization

Agencies

- Moat = repeatable creative quality at scale

Future-Proofing the Moat (Critical)

Later, CRUSHED can:

- Train **private LoRAs per brand**
- Offer **agency-locked creative intelligence**
- Become an internal creative OS

At that point:

- Switching costs become psychological + operational
- CRUSHED stops being a tool and becomes infrastructure

Final Moat Recommendation (Clear & Direct)

Use a hybrid model stack:

- 🧠 Third-party APIs for frontier quality
- 🛠️ Open-source + LoRAs for creative alignment
- 🧩 Proprietary intent-to-prompt orchestration as the core IP

Build the brain, rent the muscles.

Technical Architecture & Stack Selection

Technical Architecture & Stack Selection for CRUSHED

A) Frontend: High-performance canvas (Design + Storyboard + Ad Layout)

What the canvas must do (based on personas)

- **Designers**: drag/drop layers, text styles, masks, image transforms, export presets (1:1, 4:5, 9:16)
- **Video producers/storyboarders**: frame strip / timeline-lite, shot cards, annotations, safe zones
- **Performance marketers**: rapid resizing, templated variants, batch exports
- **Everyone**: *fast*, zero lag, works on mid laptops

Recommendation: Konva.js (2D Canvas) + optional WebGL compositor

Why Konva wins for CRUSHED

- Built for **interactive 2D scene graphs** (layers, groups, transforms, hit-testing)
- Very good dev ergonomics for “Canva-like” interactions
- Stable, mature ecosystem, predictable performance
- Easier than Fabric for complex layer/group transforms + selection UX

Default Stack

- **React + Konva (react-konva)**
- **OffscreenCanvas** for heavy ops (where supported)
- **WASM image ops** for client-side previews (resize, blur, background remove previews, etc.)

When to go WebGL

If you need **real-time effects** and very large canvases:

- Use **PixiJS** (WebGL) as a dedicated compositor for:
    - shader effects (glow, grain, chromatic aberration)
    - LUT previews
    - high FPS pan/zoom on huge artboards

Pragmatic approach

- Phase 1: **Konva-only**
- Phase 2: Add **Pixi renderer for effect layers** (hybrid)

Collaboration (optional but powerful moat)

If you want “Figma-lite” multi-user:

- **Yjs** for real-time state sync (CRDT)
- WebSocket gateway for sessions
- Store deltas + snapshots

B) Backend: Microservices for heavy processing + orchestration

Guiding principles (your UVP + moat)

- “Chat-first” means: **low latency for iteration**
- “High fidelity system prompts” means: **deterministic pipelines**
- “Multimodal outputs” means: **job-based rendering**, not synchronous HTTP

Recommended Backend Architecture (microservices + queues)

1) API Gateway (Edge)

- Routes requests, rate limits, auth, request signing
- Suggested: **Cloudflare / Fastly / AWS API Gateway**
- Supports streaming for chat responses

2) Core services (CPU-friendly)

- **Auth & Org Service**: JWT/OAuth, RBAC, roles (designer, editor, viewer)
- **Project Service**: canvases, timelines, versions, templates
- **Asset Service**: uploads, transcoding requests, metadata
- **Prompt Orchestrator Service (Your Moat)**:
  - converts conversation → structured intent
  - generates system prompts/JSON recipes
  - picks model + pipeline + constraints
- **Job Orchestrator**:
  - creates render jobs
  - pushes to queue
  - tracks status + retries

3) Worker services (heavy compute)

- **Image Processing Workers (CPU/GPU mixed)**
  - resizing, format conversion, smart crops, upscaling, background removal
- **Image Generation Workers (GPU)**
  - diffusion / style adapters / LoRAs
- **Video Generation Workers (GPU)**
  - shot generation, interpolation, compositing, encoding
- **Render/Export Workers**
  - final exports: MP4, GIF, PNG sequences, PDF boards

Data & Infra Picks

- **DB (relational): Postgres** (projects, prompts, versions, billing)
- **Cache/State: Redis** (job states, session cache, rate limiting)
- **Object Storage: S3-compatible** (assets, outputs, previews)

- **CDN:** Cloudflare/CloudFront (serve previews fast)
- **Queue:**
  - MVP: **SQS / Redis Streams**
  - Scale: **Kafka** (if you need massive throughput + ordering)
- **Search/Vector:** Postgres pgvector (MVP) → dedicated vector DB later

Why microservices (not monolith)

Because your heaviest load is **async GPU work**, and you'll want:

- separate deploy cadence for workers vs app
- per-service scaling (chat ≠ video)
- clean separation between orchestration IP and commodity compute

---

C) AI Infrastructure: Inference strategy + cold start handling

Reality check

You'll likely need **two compute modes**:

1. **Always-on baseline capacity** for fast UX
2. **Burst capacity** for spikes + long video jobs

Recommended Strategy: Hybrid

- **Dedicated GPU pool** for:
  - most common workloads
  - "instant preview" generation
  - low-latency chat + prompt orchestration + light image gen
- **Serverless GPUs** (RunPod/Modal/others) for:
  - burst traffic
  - long-running video renders
  - experimental models

This fits your moat: **own the orchestration + adapters**, rent the elastic compute.

Cold Starts: How to keep chat + generation feeling instant

Cold starts happen at:

- container start
- model download
- model load to VRAM
- compilation (TensorRT / kernels)
- first inference warmup

Mitigation Playbook (stacked defenses)

1) Warm Pools

Keep a minimum number of GPU instances warm per “tier”:

- Tier A: fast image gen (most used)
- Tier B: video / heavier pipelines
- Tier C: experimental models (cold OK)

Autoscale above baseline.

2) Model Residency Strategy

Pin the top models in VRAM on warm instances.

- Keep **1–2 “default” LoRA bundles** preloaded
- Cache weights on **local NVMe** so restart doesn’t pull from network

3) Two-stage generation UX

To kill perceived latency:

- **Stage 1 (fast):** produce a quick preview (lower steps / smaller res)
- **Stage 2 (refine):** upscale + detail pass + final render

Users feel instant progress even if the final job takes longer.

4) Batching + streaming

- For text (prompt synth / copy): stream tokens immediately
- For image: stream job events (queued → running → preview → finalize)
- For GPU utilization: batch compatible requests (where possible)

5) Routing + fallback

Inference Router chooses:

- dedicated pool if warm
- else serverless GPU
- else degrade gracefully (smaller model / preview-only)

6) Prewarming triggers

Use predictive warmup:

- based on time-of-day traffic
- based on queue depth
- based on new campaign launches (enterprise clients)

Suggested Concrete Stack (practical MVP → scale)

Frontend

- Next.js (App Router) + React
- Konva + react-konva

- Zustand (state) + Yjs (optional collab)
- Web Workers + OffscreenCanvas for previews
- WASM image utils (for fast client previews)

Backend

- Node/TypeScript (API + orchestration) or Go (if you prefer hard perf)
- Postgres + Redis
- S3 + CDN
- Queue: SQS/Redis Streams (MVP), Kafka later
- WebSocket service for job updates

AI / Workers

- Containerized GPU workers
- Inference server choices:
  - Text: vLLM / TensorRT-LLM (when needed)
  - Diffusion: optimized pipelines, keep weights local, preload LoRAs
- Hybrid compute:
  - Dedicated pool for latency
  - Serverless GPUs for burst/video

The “CRUSHED Moat” Placement in Architecture

Your moat services are:

1. **Intent Engine** (conversation → creative brief graph)
2. **Prompt Compiler** (brief graph → structured multimodal system prompt JSON)
3. **Inference Router** (cost/latency/quality routing + fallbacks)
4. **Quality Loop** (scoring + regeneration + variation logic)

Everything else is commodity infra.

Architecture Diagram Spec

Architecture Diagram Spec (Boxes + Arrows + Stores + Queues)

1) Client Layer

[Web App (Next.js)]

- Chat UI (brief Q&A)
- Canvas/Storyboard Editor (Konva / Pixi optional)
- Asset Uploader
- Job Monitor (progress states)
- Export/Download UI

Arrows

- Web App → **[Edge/API Gateway]** (HTTPS)
- Web App ⇄ **[Realtime Gateway]** (WebSocket/SSE for job updates)

---

2) Edge + Security Layer

[CDN + Edge (Cloudflare/CloudFront)]

- Static assets, previews, cached outputs
- Edge auth checks (optional)
- Rate limiting (basic)

[API Gateway]

- Auth routing, rate limits, request signing, routing to services
- Streaming support for chat responses

Arrows

- CDN ↔ Web App (static)
- Web App → API Gateway (API calls)
- API Gateway → Core Services (internal)
- API Gateway → Object Storage (signed URLs for upload/download)

---

3) Core Application Services (CPU microservices)

[Auth & Org Service]

- Users, orgs, roles, plans, quotas

[Project Service]

- Projects, canvases, timelines, versions

[Asset Service]

- Asset metadata, ingestion triggers, checksum/dedupe

[Intent Engine (Moat)]

- Conversation state → structured creative brief graph
- Persona detection (marketer/designer/video/etc.)
- Extract constraints (format, platform, brand tone)

[Prompt Compiler (Moat)]

- Brief graph → JSON “generation recipe”
- Model/pipeline selection + constraints
- Variation plan (A/B variants, aspect ratios, hooks)

[Job Orchestrator]

- Creates jobs, splits into steps (preview → refine → export)
- Retries, idempotency keys
- Writes status to Redis + Postgres
- Publishes tasks to queues

[Billing/Quota Service]

- Tracks GPU seconds, tokens, storage
- Enforces plan limits, throttling policies

Arrows

- API Gateway → Auth/Project/Asset/Intent/Prompt/Job/Billing services
- Intent Engine → Prompt Compiler → Job Orchestrator
- Job Orchestrator → **[Queues]**
- Job Orchestrator → Redis (job status)
- All core services → Postgres (source of truth)

---

4) Data Stores

[Postgres]

- Users/orgs/projects
- Conversation snapshots
- Prompt recipes + versioning
- Job metadata + audit logs

[Redis]

- Job state cache (queued/running/progress)
- Rate limit counters
- Session cache for chat
- Short-lived orchestration state

[Object Storage (S3/R2/GCS)]

- Raw uploads (images, refs, videos)
- Generated artifacts (previews, finals)
- Model artifacts (optional), LoRA bundles (if stored)

[Vector Store (pgvector initially)]

- Retrieval for:
    - prompt patterns
    - style anchors
    - brand guidelines
    - past successful outputs
    - reference matching

Arrows

- Asset Service → Object Storage (store) + Postgres (metadata)
- Workers → Object Storage (write outputs) + Postgres (finalize metadata)
- Intent/Prompt services → Vector store (RAG retrieval) + Postgres (persistence)

---

5) Queues (Async Backbone)

[Queue: “jobs.control”]【SQS/Redis Streams/Kafka】

- job.created
- job.cancelled
- job.retry

[Queue: “tasks.image”]【GPU-bound】

- image.generate.preview
- image.generate.refine
- image.upscale
- image.bgremove
- image.style.apply (LoRA)

[Queue: “tasks.video”]【GPU/long-running】

- video.generate.shots
- video.interpolate
- video.compose
- video.encode

[Queue: “tasks.export”]【CPU-heavy】

- export.png.batch
- export.mp4
- export.pdf.storyboard

Arrows

- Job Orchestrator → queues (publish)
- Workers ← queues (consume)
- Workers → Redis/Postgres (status updates)
- Workers → Realtime Gateway (events) OR Job Orchestrator pushes updates

---

6) Worker Plane (Compute)

[CPU Worker Pool]

- transcode, thumbnails, OCR (if ever), format conversions
- PDF/board exports, zips, batch ops

[GPU Worker Pool – Dedicated Warm]

- most common image generation + fast preview
- top models pinned in VRAM + popular LoRAs preloaded

[GPU Worker Pool – Serverless Burst] (RunPod/Modal-like)

- overflow traffic
- long video jobs
- experimental models

[Inference Router (Moat-adjacent)]

- Chooses execution target:
  - Dedicated GPU if warm/available
  - Burst GPU if queue depth high
  - Degrade to preview-only or smaller model under load
- Maintains “warm pool” health + model residency map

Arrows

- Job Orchestrator → Inference Router (optional) OR router embedded within orchestrator
- Inference Router → dispatches tasks to Dedicated GPU queue or Burst GPU queue
- Workers → Object Storage + Redis/Postgres
- Workers → Realtime Gateway (progress events)

---

7) Realtime + Observability

[Realtime Gateway (WebSocket/SSE)]

- streams job state updates to client

[Observability Stack]

- Logs: ELK/Cloud provider

- Metrics: Prometheus/Grafana
- Tracing: OpenTelemetry
- Error tracking: Sentry

Arrows

- Core services + workers → Observability
- Workers/Orchestrator → Realtime Gateway → Web App

Deployment Topology

Deployment Topology (What Runs Where)

A) Edge / Global

Cloudflare/CloudFront + API Gateway

- static + previews cached at edge
- signed URL issuance for uploads/downloads
- request throttling + WAF rules

Scaling

- automatic at edge
- cost lever: cache TTLs for previews & popular outputs

B) Core Services Cluster (CPU)

Kubernetes (EKS/GKE/AKS) or ECS
Runs:

- API services (Auth, Project, Asset, Intent, Prompt, Job, Billing)
- Realtime Gateway
- Lightweight “text inference” if you self-host small LLMs (optional)

Scaling rules

- HPA on CPU + request rate
- separate scaling for:
    - Chat/Intent/Prompt (latency-sensitive)
    - Project/Asset (throughput)
- prioritize streaming endpoints capacity

Cost levers

- aggressive autoscaling down at low traffic
- reserve instances for steady baseline
- multi-tenant org isolation by RBAC, not by infra (cheaper)

C) Data Layer

Managed services strongly recommended

- Postgres: RDS/CloudSQL
- Redis: Elasticache/Memorystore
- Object storage: S3/R2
- Queue: SQS/PubSub (MVP) → Kafka later if needed

Scaling rules

- Postgres: read replicas for dashboards/analytics
- Redis: keep small but monitor memory; use TTLs aggressively
- Storage: lifecycle policies (move old assets to cheaper tier)

Cost levers

- store only necessary intermediates (delete raw previews after final)
- dedupe assets by checksum
- compress + convert outputs to efficient formats

D) Worker Plane (Compute)

1) Dedicated GPU Pool (Warm, low-latency)

Kubernetes GPU node group (or dedicated GPU instances)
Runs:

- image gen workers
- “fast preview” endpoints
- most used models pinned in VRAM
- LoRA bundles cached on local NVMe

Scaling rules

- Keep **minimum warm replicas** per tier:
  - Tier A: Image Preview (always-on)
  - Tier B: Image Refine/Upscale (low warm)
  - Tier C: Video (maybe 0 warm if you can tolerate queueing)
- Scale up on:
  - queue depth
  - GPU utilization %
  - job wait time SLA

Cost levers

- cap max replicas per org/plan
- prioritize premium customers in routing
- precompute “preview” using fewer steps to cut GPU time

2) Burst GPU Plane (Serverless)

RunPod/Modal-style ephemeral GPUs
Runs:

- overflow image jobs
- long video generation tasks
- experimental pipelines

Scaling rules

- spawn when dedicated queue depth exceeds threshold or ETA > X seconds
- terminate aggressively when idle
- route only non-latency-critical or long jobs here

Cost levers

- use spot/preemptible where safe (retryable jobs)
- enforce max runtime per job + checkpoint outputs
- gate expensive pipelines behind plan tiers

3) CPU Worker Pool

K8s or serverless (Cloud Run/Lambda where viable)
Runs:

- exports, packaging, encoding (if CPU-based)
- format conversions
- metadata extraction

Scaling rules

- scale on queue depth
- prioritize export completion for UX satisfaction

Cost levers

- move CPU-heavy encode to cheaper compute
- batch exports to reduce overhead
- avoid storing massive intermediates

Cost-Control Levers

Cost-Control Levers (The Ones That Actually Move the Needle)

1. **Tiered Inference Routing**

- Free/Basic: preview-only + burst GPUs
- Pro: dedicated GPU priority
- Enterprise: reserved capacity + brand LoRAs

2. **Quality Ladder**

- default: cheap preview first
- only run expensive refine if user accepts preview direction

3. **Quota by GPU-seconds, not “generations”**

- aligns pricing with cost
- reduces abuse

4. **Cache & Reuse**

- prompt recipe + seed + LoRA = reproducible outputs
- avoid re-running identical requests

5. **Batching**

- batch variant generation for marketers (same pipeline, different copy/layout)

6. **Delete Intermediates**

- lifecycle old previews + intermediate frames
- keep finals + minimal audit artifacts

Core AI Feature Engineering

Core AI Feature Engineering (CRUSHED)

A) Unified Pipeline for Text + Image + Video (with ControlNet + Upscaling)

0) Design goals (derived from UVP + personas)

- **Chat-first**: user speaks in intent; system asks brief questions.
- **High fidelity**: outputs must be “commercial-ready,” not “AI-demo-ready.”
- **Determinism & traceability**: every output is reproducible (recipe versioning + seeds + lineage).
- **Multimodal continuity**: one brief drives copy, key visuals, storyboard frames, and video shots.
- **Routing moat**: best model per task, per persona, per cost/latency/quality tier.

A1) End-to-end flow (Conversation → PromptRecipe → Jobs)

Stage 1 — Intent Capture (Chat)

Inputs

- Natural language brief
- Persona detection (marketer / designer / video producer / storyboard)
- Constraints: platform (IG/YouTube), formats, brand tone, target audience, “must-have / avoid”
- References: moodboard, product shots, brand kit, existing ads

Process

1. **Intent Engine** builds a **Creative Brief Graph**:
    - objective (performance vs brand)
    - visual style anchors
    - narrative arc (if video)
    - compliance constraints (logo usage, product fidelity, prohibited themes)
2. **Question Sequencer** asks only the missing info (adaptive QnA):
    - “Is this performance-first or brand-first?”
    - “Do we need negative space for text? Where?”
    - “Is the product shot mandatory? Can we stylize it?”

Outputs

- `BriefGraph` (structured)
- `PersonaProfile` (used for defaults)
- `Constraints` (format + layout + brand-safe)

Stage 2 — Prompt Compiler (Your core IP)

Turns the `BriefGraph` into a **PromptRecipe JSON** (versioned).

PromptRecipe JSON contains

- `modality`: text | image | video | storyboard
- `model_plan`: ordered calls (LLM → image → post → export)
- `prompt_blocks`: system prompt + user prompt + negative prompt policy
- `control_plan`: ControlNet configs (if any)
- `lora_plan`: LoRA bundles + weights (if OSS pipelines used)
- `render_plan`: resolution, aspect ratios, variants, seed policy
- `post_plan`: upscaling/restoration decisions
- `cost_plan`: max GPU seconds, tier constraints

CRUSHED is “model agnostic,” but **recipe strict**.

Stage 3 — Job Graph Execution (Async, queue-based)

The Job Orchestrator expands the recipe into tasks.

Job graph (canonical)

1. `prepare_inputs` (CPU)
2. `retrieve_refs` (Vector + storage) (CPU)
3. `generate_control_maps` (CPU)
4. `run_text_tasks` (LLM) (CPU)
5. `run_image_tasks` (GPU) (preview → refine)
6. `run_video_tasks` (GPU) (shots → consistency → encode)
7. `postprocess` (GPU/CPU) (Real-ESRGAN / GFPGAN / color ops)
8. `export_variants` (CPU/GPU)
9. `publish_outputs` (metadata + CDN)

Progress streams via WebSocket/SSE.

A2) Model Integration: a unified “Model Adapter Layer”

Why you need it

You’re integrating:

- **Third-party APIs** (GPT-5.2, Sora 2, Gemini → Imagen 4/Nano Banana Pro/Veo 3.1, Seedream/Seedance)
- **Self-hosted OSS** (SDXL/FLUX/Wan + ControlNet + LoRAs)
- **Post-processing models** (GFPGAN/Real-ESRGAN)

So you implement a **Model Adapter Interface** so orchestration stays stable even if providers change.

Adapter interface (conceptual)

Each model adapter implements:

- `capabilities()` (supports image/video? img2img? ref images? max res? latency class?)
- `estimate_cost()` (per second/token/call)
- `prepare_request(recipe, inputs)`
- `invoke(request)` (sync/async)
- `normalize_output()` (common schema: urls, thumbnails, metadata)
- `retry_policy()` (idempotency + backoff + failover chain)

Result: your orchestration + UI doesn't care "which model," only "what capability."

A3) Routing Strategy (persona-aware + modality-aware)

Text Brain (Intent + Copy + Script)

Primary: GPT-5.2
Secondary: Gemini (long docs / alt reasoning)
OSS cost tier: Gemma / LLaMA family fine-tunes for bulk copy and tonal presets

Typical text tasks

- Ad hooks & variations (performance marketers)
- Captions, headlines, CTAs
- Script outlines + shot lists (video producers)
- Storyboard narration + scene descriptions
- PromptRecipe generation (JSON)

Key rule

- The "Prompt Compiler" must be deterministic: same brief + same version → stable recipe.

Image Engine (Key visuals / thumbnails / ads)

Premium realism: Imagen 4
Fast iteration: Nano Banana Pro
Storyboard frames / conceptual coherence: Sora 2 (frames)
Aesthetic social-first: Seedream
Layout + spatial control + brand LoRAs: SDXL / Wan / FLUX (self-hosted)

Routing triggers

- Needs strict layout / negative space → OSS (SDXL/Wan + ControlNet)
- Needs photoreal product & lifestyle → Imagen 4
- Needs many variants quickly → Nano Banana Pro
- Needs narrative keyframes with continuity → Sora 2 frames (+ optional OSS refinement)
- Needs Gen-Z aesthetic poster vibe → Seedream

Video Engine (Reels / DVC / cinematic)

Cinematic premium: Veo 3.1
Narrative continuity / previsualization: Sora 2
Motion-heavy vibes: Seedance

Tooling/iteration + accessible pricing tiers: Runway/Pika/Luma (optional)
OSS control pipelines: Wan video / Stable Video Diffusion (optional, advanced)

Routing triggers

- "Brand film look" + realism → Veo 3.1
- "Pitch / storyboard" + story coherence → Sora 2
- "Kinetic dance/action" → Seedance
- "Cheap fast drafts" → lower-cost providers (optional)

A4) ControlNet for Spatial Consistency (CRUSHED-critical)

What ControlNet solves in CRUSHED

- Marketers want **text safe zones**
- Designers want **composition repeatability**
- Video/storyboard wants **shot-to-shot framing consistency**
- Product creatives want **angle + silhouette preservation**

Control inputs CRUSHED should support

1. **Canvas-derived layout guide**

- From Konva scene:
  - safe zones
  - subject box
  - "no-text" regions
  - rough mask layers
- Generate a **layout mask** image used as conditioning.

2. **Reference-derived control maps**
   Generated via CPU workers, cached by hash:

- Canny edges (products/objects)
- Depth map (rooms/environments)
- Pose (OpenPose) for humans
- Segmentation (foreground/background separation)
- Lineart for illustration style

3. **Storyboard frame guide**

- For multi-frame consistency:
  - control maps reused across frames with slight variations
  - seed strategy + "anchor frames"

ControlNet configuration stored in PromptRecipe

- `control_type`: canny | depth | pose | segmentation | lineart | layout_mask
- `strength`: 0.1–1.0

- `start_step / end_step` to stage influence
- `resize_policy`: pad/fit/crop to preserve composition
- `frame_locking`: for storyboards/videos (optional)

Where ControlNet applies

- **Self-hosted OSS image pipelines** (SDXL/Wan/FLUX)
- For third-party APIs that support "guided generation," you use:
  - image conditioning / reference image / composition guides where available
  - otherwise, you do "hybrid": generate base with API → align/refine with OSS + ControlNet

---

A5) Finishing Stack: Real-ESRGAN + GFPGAN (and when to use them)

Real-ESRGAN (default upscale)

Used for

- Ad-ready sharpness, improved micro-detail
- Meeting platform export specs (e.g., 1080×1350, 4K)

Policy

- Always run if output < target resolution OR user requests "print/4K"
- Use x2 first; x4 only when needed
- Keep "stylized" safeguard: reduce strength on art styles that artifact

GFPGAN (conditional face restoration)

Used for

- Fixing distorted faces, improving realism

Policy

- Only run if face detection confidence is high AND distortion score triggers
- For stylized outputs, prefer **skip or low-strength**
- Optionally expose "face fidelity slider" (internal) while UX stays simple

Output guarantee

Final step emits:

- `final_url`, `preview_url`, `thumb_url`
- full provenance: `recipe_id`, `seed`, `model_hashes`, `lora_ids`, `control_maps_ids`

A6) Multi-variant generation (Marketer-first feature)

For performance marketers, CRUSHED should generate:

- 10–30 variants *without breaking composition*

How

- Lock:
  - seed (or use seed family)
  - ControlNet maps
  - layout mask
- Vary:
  - hook copy text layer (done in canvas, not inside image model)
  - minor scene elements (props, color temperature)
  - background style intensity
- Export batch with naming conventions and metadata for ads manager

---

B) Asset Management Strategy (S3 + CDN + Vector Search)

B1) Storage layout (S3/R2) — one bucket per environment

Bucket naming:

- `crushed-prod-assets`
- `crushed-stage-assets`
- `crushed-dev-assets`

Prefix strategy (multi-tenant):

- `org/{org_id}/project/{project_id}/uploads/`
- `org/{org_id}/project/{project_id}/derived/` (control maps, thumbnails, masks)
- `org/{org_id}/project/{project_id}/previews/`
- `org/{org_id}/project/{project_id}/final/`
- `org/{org_id}/project/{project_id}/exports/` (zip, pdf boards, mp4 sets)
- `org/{org_id}/brandpacks/{brandpack_id}/` (guidelines, palettes, fonts)
- `org/{org_id}/models/lora/{lora_id}/` (if storing LoRAs)

Asset metadata schema (stored in Postgres)

For each asset:

- `asset_id`, `org_id`, `project_id`, `type`
- `sha256` (dedupe key)
- `mime`, `width`, `height`, `duration`, `fps`
- `source_asset_ids` (lineage)
- `recipe_id`, `job_id`, `task_id`
- `tags` (platform, campaign, vibe)

- `retention_class` (preview/derived/final)

B2) CDN distribution (fast previews, secure finals)

Upload

- Client uploads directly via **signed PUT** (no app server bottleneck)

Read / Delivery

- Previews and finals served via **CDN**:
  - public caching for share links (optional)
  - token-auth or signed URL for private enterprise assets

Caching rules (simple and effective)

- `previews/*`: short TTL (minutes–hours)
- `final/*`: long TTL (days–weeks)
- Use **content-hash filenames** to avoid cache invalidation

B3) Vector search (pgvector first, evolve later)

What you embed

Text embeddings

- BriefGraph summary
- PromptRecipe (human-readable + JSON features)
- Captions/headlines/hooks
- Brand guidelines / tone

Image embeddings

- Moodboards
- Generated outputs
- Key reference images
- Product shots (for similarity & reuse)

Video embeddings (optional early)

- Shot descriptions + thumbnails embeddings
- Keyframe embeddings (later)

Retrieval patterns CRUSHED needs

- “Find outputs like this vibe”
- “Reuse the style from last campaign”
- “Show me successful thumbnails for X niche”

- “Find all visuals with ‘premium minimal kitchen’ tone”

Hard multi-tenant safety

Always filter retrieval by:

- `org_id` mandatory
- optionally `project_id`
- plus policy filters (brandpack constraints)

B4) Lifecycle + cost governance (must-have)

Lifecycle policies

- `derived/*` control maps: delete after 30–90 days (plan-based)
- `previews/*`: keep 7–30 days
- `exports/*`: expire quickly unless saved
- `final/*`: retained longer (plan-based)

Dedupe

- Use SHA-256 to prevent duplicate uploads and repeated intermediate saves

Provenance retention

- Keep PromptRecipe + seed + minimal thumbnails even if previews expire
  (this keeps your “creative memory” moat while controlling storage costs)

The CRUSHED Moat, expressed technically

CRUSHED becomes hard to copy because you store and learn from:

- **BriefGraph** → **PromptRecipe** → **Output** lineage
- Control maps + layout masks + LoRA selections
- Which routing decisions produced the best outcomes per persona

That’s your compounding dataset—without building foundation models.

Inference Router

CRUSHED — Inference Router

Model Selection + Fallback Decision Tree (Executable Pseudo-Logic)

---

0. Core Principles (Hard Rules)

```
RULE 1: Intent > Modality > Fidelity > Cost > Latency
RULE 2: Always return *something* (preview-first)
RULE 3: Never block chat on GPU availability
RULE 4: Prefer determinism for paid tiers
RULE 5: OSS + ControlNet when spatial control is required
```

---

1. Canonical Inputs to the Router

```
Input {
  persona: "marketer" | "designer" | "video_producer" | "storyboard" | "freelancer" | "casual",
  modality: "text" | "image" | "video" | "storyboard",
  intent: {
    goal: "performance" | "branding" | "pitch" | "exploration",
    realism: "high" | "medium" | "stylized",
    narrative: boolean,
    spatial_constraints: boolean,
    product_fidelity_required: boolean
  },
  constraints: {
    layout_mask?: boolean,
    text_safe_zones?: boolean,
    reference_images?: boolean,
    face_priority?: boolean,
    platform?: "IG" | "YT" | "Ads" | "Print"
  },
  tier: "free" | "pro" | "enterprise",
  latency_budget_ms,
  cost_budget_units,
  gpu_state: {
    dedicated_available: boolean,
    warm_models: string[],
    queue_depth: number
  }
}
```

---

2. High-Level Router Skeleton

```
function routeInference(input) {
  if (input.modality === "text") {
    return routeText(input)
  }

  if (input.modality === "image") {
    return routeImage(input)
  }

  if (input.modality === "video") {
    return routeVideo(input)
  }

  if (input.modality === "storyboard") {
    return routeStoryboard(input)
```

```
  }

  throw new Error("Unsupported modality")
}
```

---

3. TEXT ROUTING (Intent + Copy + Scripts)

```
function routeText(input) {

  // Tier-based hard preference
  if (input.tier === "enterprise") {
    return GPT_5_2_PRIMARY()
  }

  // Complex reasoning or long docs
  if (input.intent.narrative || input.constraints.reference_images) {
    return GPT_5_2_PRIMARY()
  }

  // Bulk, cost-sensitive copy
  if (input.cost_budget_units < LOW_COST_THRESHOLD) {
    return GEMMA_FINE_TUNED()
  }

  // Fallback
  return GEMINI_PRO_SECONDARY()
}
```

Text Fallback Chain

```
GPT_5_2 → Gemini → Claude → OSS (Gemma / LLaMA)
```

---

4. IMAGE ROUTING (Key Visuals, Ads, Thumbnails)

4.1 First: Check Spatial Control Requirement

```
function routeImage(input) {

  if (input.intent.spatial_constraints === true ||
      input.constraints.layout_mask === true ||
      input.constraints.text_safe_zones === true) {

    return routeImageWithControl(input)
  }

  return routeImageFreeform(input)
}
```

4.2 Image Routing WITH Spatial Constraints (ControlNet Path)

```
function routeImageWithControl(input) {

  // ControlNet requires OSS pipeline
  if (gpuAvailableForOSS()) {
    return {
      base_model: SDXL_OR_WAN(),
      controlnet: ENABLED,
      loras: selectLoRAs(input),
      post: postPipeline(input)
    }
  }
```

```
  // OSS GPU unavailable → degraded hybrid fallback
  return {
    base_model: IMAGEN_4_API(),
    refinement: OSS_CONTROLNET_PASS(), // optional secondary job
    post: postPipeline(input)
  }
}
```

Why:
Commercial layouts **must not break**. OSS + ControlNet is non-negotiable here.

4.3 Image Routing WITHOUT Spatial Constraints (Freeform)

```
function routeImageFreeform(input) {

  // Performance marketing / fast variants
  if (input.intent.goal === "performance") {
    return {
      base_model: NANO_BANANA_PRO(),
      variants: HIGH,
      post: postPipeline(input)
    }
  }

  // High realism + product fidelity
  if (input.intent.realism === "high" ||
      input.intent.product_fidelity_required) {
    return {
      base_model: IMAGEN_4_API(),
      post: postPipeline(input)
    }
  }

  // Narrative / conceptual visuals
  if (input.intent.narrative === true) {
    return {
      base_model: SORA_2_IMAGE_FRAMES(),
      post: postPipeline(input)
    }
  }

  // Aesthetic-first / social visuals
  if (input.persona === "designer" || input.persona === "freelancer") {
    return {
      base_model: SEEDREAM(),
      post: postPipeline(input)
    }
  }

  // Default safe fallback
  return {
    base_model: SDXL(),
    post: postPipeline(input)
  }
}
```

---

5. VIDEO ROUTING (Reels, DVCs, Cinematic)

```
function routeVideo(input) {

  // Storyboard / pitch mode
```

```
if (input.modality === "storyboard" ||
    input.intent.narrative === true) {

  return {
    base_model: SORA_2_VIDEO(),
    output: "keyframes + low-fps motion"
  }
}

// Premium brand film
if (input.intent.realism === "high" &&
    input.tier !== "free") {

  return {
    base_model: VEO_3_1(),
    compute: LONG_GPU_JOB,
    post: VIDEO_POST_PIPELINE()
  }
}

// Motion-first / kinetic content
if (input.intent.goal === "branding" &&
    input.persona === "designer") {

  return {
    base_model: SEEDANCE(),
    post: VIDEO_POST_PIPELINE()
  }
}

// Cost-sensitive / fast drafts
if (input.cost_budget_units < VIDEO_LOW_COST_THRESHOLD) {
  return {
    base_model: RUNWAY_OR_PIKA(),
    post: BASIC_POST()
  }
}

// OSS fallback
return {
  base_model: WAN_VIDEO(),
  post: VIDEO_POST_PIPELINE()
}
}
```

Video Fallback Chain

`Veo → Sora → Seedance → Runway/Pika → Wan Video`

---

6. POST-PROCESSING PIPELINE (Shared)

```
function postPipeline(input) {

  const steps = []

  // Upscaling
  if (input.constraints.platform === "Print" ||
      input.intent.realism === "high") {
    steps.push(REAL_ESRGAN_X2())
  }

  // Face restoration (conditional)
  if (input.constraints.face_priority === true) {
```

```
    steps.push(GFPGAN_CONDITIONAL())
  }

  // Variant export
  steps.push(EXPORT_VARIANTS(input.constraints.platform))

  return steps
}
```

---

7. GPU AVAILABILITY + COLD START HANDLING

```
function gpuAvailableForOSS() {
  return (
    gpu_state.dedicated_available === true ||
    gpu_state.queue_depth < MAX_QUEUE_THRESHOLD
  )
}
```

If GPU is cold/unavailable

```
if (!gpuAvailableForOSS()) {
  return {
    immediate_preview: LOW_RES_API_MODEL(),
    deferred_refine: OSS_PIPELINE_WHEN_WARM()
  }
}
```

---

8. Global Fallback Policy (Never Fail)

```
try {
  runPrimaryPipeline()
} catch (error) {

  if (error.isTimeout || error.isQuotaExceeded) {
    runNextBestModel()
  }

  if (allFail) {
    return {
      output: "preview_only",
      message: "Refining output…"
    }
  }
}
```

---

9. Determinism & Reproducibility Guard

```
if (tier !== "free") {
  lock_seed = true
  lock_recipe_version = true
}
```

Free tier:

- relaxed determinism
- fewer steps
- cheaper routing

Inference Router - Visual Flow Diagram

Visual Flow Diagram (System-Level)

```
┌──────────────────────────────────┐
│        USER CHAT INPUT           │
│ (Brief, refs, constraints)       │
└──────────────┬───────────────────┘
               ↓
┌──────────────────────────────────┐
│        INTENT ENGINE             │
│ Persona + Goal + Constraints     │
└──────────────┬───────────────────┘
               ↓
┌──────────────────────────────────┐
│       PROMPT COMPILER            │
│  (PromptRecipe JSON vX.Y)        │
└──────────────┬───────────────────┘
               ↓
┌──────────────────────────────────┐
│        INFERENCE ROUTER          │
│  (Decision Tree + Policies)      │
└──────┬──────────┬──────────┬─────┘
       ↓          ↓          ↓
   TEXT FLOW  IMAGE FLOW  VIDEO FLOW
```

1️⃣ TEXT ROUTING FLOW (Creative Reasoning & Copy)

```
┌──────────────────────────────────┐
│        MODALITY = TEXT           │
└──────────────┬───────────────────┘
               ↓
┌──────────────────────────────────┐
│ Complex reasoning / narrative    │
│ OR references present?           │
└──────┬──────────────┬────────────┘
       │YES           │NO
       ↓              ↓
┌──────────────┐   ┌──────────────────┐
│   GPT-5.2    │   │ Cost-sensitive?  │
│ (Primary)    │   └──────┬───────────┘
└──────────────┘          │YES
                          ↓
                   ┌──────────────────┐
                   │ Gemma (Fine-tuned)│
                   └──────┬───────────┘
                          │NO
                          ↓
                   ┌──────────────────┐
                   │ Gemini / Claude  │
                   │ (Secondary)      │
                   └──────────────────┘
```

Fallback chain (implicit):

```
GPT-5.2 → Gemini → Claude → OSS (Gemma / LLaMA)
```

2️⃣ IMAGE ROUTING FLOW (Key Visuals, Ads, Thumbnails)

A. First Decision: Spatial Control Needed?

```
┌──────────────────────────────────┐
│        MODALITY = IMAGE          │
└──────────────────┬───────────────┘
                   ↓
┌──────────────────────────────────┐
│ Layout mask / text safe zone     │
│ or spatial consistency req?      │
└──────────┬──────────────────┬────┘
           │YES               │NO
```

B. IMAGE WITH SPATIAL CONSTRAINTS (ControlNet Path)

```
          YES
           ↓
┌──────────────────────────────────┐
│ OSS GPU available?               │
└──────────┬──────────────────┬────┘
           │YES               │NO
           ↓                  ↓
┌──────────────────────┐   ┌──────────────────────┐
│ SDXL / WAN +         │   │ Imagen 4 (API)       │
│ ControlNet + LoRA    │   │ + Deferred OSS       │
└──────────────────────┘   │   Refinement         │
                           └──────────────────────┘
```

Why: Layout fidelity > aesthetic freedom.

C. IMAGE WITHOUT SPATIAL CONSTRAINTS (Freeform)

```
          NO
           ↓
┌──────────────────────────────────┐
│ Intent = Performance?            │
└──────────┬──────────────────┬────┘
           │YES               │NO
           ↓                  ↓
┌──────────────────────┐   ┌──────────────────────────┐
│ Nano Banana Pro      │   │ High realism / product   │
│ (Fast variants)      │   │ fidelity needed?         │
└──────────────────────┘   └──────────┬───────────────┘
                                       │YES        │NO
                                       ↓           ↓
                           ┌──────────────────┐  ┌──────────────────┐
                           │ Imagen 4         │  │ Narrative needed? │
                           └──────────────────┘  └──────────┬───────┘
                                                            │YES
                                                            ↓
                                                  ┌──────────────────┐
                                                  │ Sora 2 (frames)  │
                                                  └──────────┬───────┘
                                                             │NO
                                                             ↓
                                                  ┌──────────────────┐
                                                  │ Seedream /       │
                                                  │ SDXL (default)   │
                                                  └──────────────────┘
```

3️⃣ VIDEO ROUTING FLOW (Reels, DVCs, Storyboards)

```
┌──────────────────────────────┐
│        MODALITY = VIDEO      │
└──────────────┬───────────────┘
               ↓
┌──────────────────────────────┐
│ Narrative / Storyboard mode? │
└──────┬──────────────┬────────┘
       │YES           │NO
       ↓              ↓
┌────────────────────┐   ┌────────────────────────┐
│ Sora 2 (Video)     │   │ Premium realism needed?  │
│ (Keyframes / flow) │   └──────┬──────────┬──────┘
└────────────────────┘          │YES       │NO
                                ↓          ↓
                         ┌────────────────┐  ┌────────────────────┐
                         │ Veo 3.1        │  │ Motion-heavy?      │
                         │ (Cinematic)    │  └──────┬─────────────┘
                         └────────────────┘         │YES
                                                    ↓
                                              ┌────────────────┐
                                              │ Seedance       │
                                              └──────┬─────────┘
                                                     │NO
                                                     ↓
                                              ┌────────────────┐
                                              │ Runway / Wan   │
                                              │ (Fallback)     │
                                              └────────────────┘
```

4️⃣ POST-PROCESSING & EXPORT LANE (Shared)

```
┌──────────────────────────────┐
│        POST PIPELINE         │
└──────────────┬───────────────┘
               ↓
┌──────────────────────────────┐
│ Resolution < target?         │
└──────┬──────────────┬────────┘
       │YES           │NO
       ↓              ↓
┌────────────────────┐          │
│ Real-ESRGAN x2/x4  │          │
└──────────┬─────────┘          │
           ↓                    │
┌──────────────────────────────┐
│ Face priority & distortion?  │
└──────┬──────────────┬────────┘
       │YES           │NO
       ↓              ↓
┌────────────────────┐          │
│ GFPGAN / CodeFormer│          │
└──────────┬─────────┘          │
           ↓                    ↓
       ┌──────────────────────────────┐
       │ Export variants (IG/YT/Ads)  │
       │ + Publish to CDN             │
       └──────────────────────────────┘
```

5️⃣ GLOBAL FAILSAFE (Always Return Something)

```
┌──────────────────────────────┐
```

```
| Primary model fails?         |
└────────┬──────────────────────┘
        |YES          |NO
        ↓             ↓
┌────────────────────┐        DONE
| Next fallback tier |
└────────┬───────────┘
        ↓
┌────────────────────┐
| Preview-only output|
| + async refinement |
└────────────────────┘
```

Diagram Legend (for designers / decks)

- **Rectangles** → Processing stages
- **Diamonds** → Decisions
- **Solid arrows** → Primary path
- **Dashed arrows** → Fallback / degraded path
- **Left branches** → Higher quality / cost
- **Right branches** → Faster / cheaper

MVP Development & Iteration (Agile)

MVP Development & Iteration (Agile)

1) The “Walking Skeleton” of CRUSHED

A walking skeleton is the **smallest end-to-end slice** that proves the product works across UI → orchestration → compute → assets → delivery, even if everything is “thin.”

Walking Skeleton Goal

A user can chat a brief → get 1 image output → iterate once → export and share.
(That’s enough to validate UVP: “intent → system prompt → output” without nodes.)

A. Walking Skeleton: End-to-End Thin Slice

WS-0: Foundations (Week 0)

Non-negotiables

- Auth (email + magic link / OAuth) + org workspace
- Project creation
- Upload reference image(s)
- Basic audit logging (job provenance)

Why: multi-tenant + traceability is part of your moat.

WS-1: Chat → PromptRecipe → Preview Image (Core)

User Flow

1. User creates project
2. User opens chat: “I need a premium IG 4:5 key visual for a cookware launch…”
3. System asks **2–4 adaptive Qs** (goal, style, layout needs, realism)
4. User hits “Generate”
5. User receives **preview output** + can regenerate

Backend/AI

- Intent Engine (thin): persona + goal + constraints classifier
- Prompt Compiler (thin): produces a **PromptRecipe JSON** with:
  - model chosen (start with 1 image model + 1 text model)
  - seed + resolution + aspect ratio
  - minimal negative prompt policy
- Job Orchestrator: async queue
- GPU worker: generates **1 image preview**
- Asset pipeline: stores preview → returns CDN URL
- Realtime updates: job status streaming

Success Criteria

- Median time to first preview < reasonable baseline for your infra

- Recipe saved + versioned (v0.1)

WS-2: Spatial Control (the “CRUSHED difference”)

User Flow

- User chooses a simple layout preset: “Headline top, product right”
- System generates with **layout mask + ControlNet** (OSS path)

Backend/AI

- Canvas-lite (not full editor yet): selectable layout presets generate a mask
- ControlNet maps generated (CPU)
- OSS pipeline (SDXL/Wan) with ControlNet enabled

Success Criteria

- Outputs reliably respect safe zones
- This is where your UVP becomes real vs generic generators

WS-3: Finishing (Commercial Readiness)

User Flow

- User clicks “Enhance & Export”
- Gets final 1080×1350 (or 4K) export

Backend/AI

- Real-ESRGAN upscale
- GFPGAN conditional face restoration (only if needed)
- Export service generates:
    - PNG/JPG
    - batch variants (optional later)
- Outputs stored as immutable hashed paths

Success Criteria

- “Client-ready” output with consistent exports
- Provenance preserved

WS-4: One Iteration Loop (Retention Hook)

User Flow

- User says: “Make it more editorial, darker mood, keep layout”
- System reuses:
    - same layout mask / ControlNet maps

    - seed-family policy (controlled variation)
- Produces 2–4 variants

Success Criteria

- Iteration feels *directed*, not random
- Users feel “I’m steering a system,” not “rolling dice”

B. MVP Scope Lock (What NOT to build yet)

To protect velocity:

- No full Canva/Figma editor
- No multi-user collab
- No video generation in v1 (unless your wedge is storyboard frames)
- No marketplace of templates
- No custom brand LoRAs for MVP (but store data to enable it later)

MVP wedge recommendation

- **Marketers + Designers** first: “ad-ready key visuals + layout control”
- Add storyboard frames next (not full video)

---

2) CI/CD Pipeline for ML Models (MLOps) — CRUSHED-Specific

You’re not “training foundation models.” You’re shipping:

- **Prompt Compiler logic** (changes often)
- **LoRAs/adapters** (periodic)
- **Inference containers** (GPU workers)
- **Model routing policies** (continuous)
- **Quality evaluators** (continuous)

Think of this as **Model Supply Chain CI/CD**.

A. Model Artifact Versioning (The Core Unit)

Everything ships as immutable artifacts:

Artifacts to version

1. **Base model reference**
    - `model_name`, `provider`, `checkpoint_hash`
2. **LoRA bundles**
    - `lora_id`, `sha256`, training dataset version, hyperparams

3. **Inference container**
    - Docker image digest with CUDA + libs pinned
4. **PromptRecipe schema version**
    - ensures reproducibility across time
5. **Routing policy version**
    - decision tree weights and thresholds

Rule: Every output stores:

- `recipe_id`
- `model_artifact_versions`
- `seed`
- `control_map_hashes`

That's your audit + replay.

B. MLOps CI/CD Stages (Pipeline)

Stage 1 — Code CI (Prompt Orchestrator + Router)

Triggered on PR merge.

- Lint, unit tests
- Contract tests for:
    - `PromptRecipe JSON schema`
    - adapter interface (Gemini/Imagen, OpenAI, OSS)
- Golden tests: same brief → stable recipe output (snapshot tests)

Deploy

- Blue/green deploy to staging
- Canary to 5–10% prod traffic

Stage 2 — Model Build CI (LoRA training + packaging)

Triggered on:

- scheduled cadence (weekly/biweekly)
- "training data ready" event
- manual release for a new style pack

Steps

1. **Dataset assembly**
    - curated ad samples + moodboards + treatments
    - dedupe, quality filters, license validation metadata

2. **Training**
    - LoRA training job (GPU)
    - logs + metrics saved
3. **Validation suite**
    - run the LoRA against a fixed **PromptSet**
    - compute quality scores (CLIP similarity, aesthetic score, human eval gates)
4. **Package artifact**
    - upload LoRA bundle to model registry (S3 + metadata DB)
    - generate `lora_manifest.json`

Promotion gate

- must pass:
    - regression thresholds
    - brand safety checks
    - “no degradation on layout fidelity” tests

Stage 3 — Inference Image Build & Deployment (GPU Workers)

Triggered on:

- new inference code
- new dependencies
- new model/LoRA manifests

Steps

- Build GPU worker image (pinned CUDA/cuDNN/torch)
- Integration tests:
    - “load model in < X seconds” (warm start)
    - “first inference under Y seconds” (warm pool)
    - ControlNet path sanity
- Push image to registry
- Deploy to:
    - staging GPU pool
    - canary GPU pool (small)
    - full production GPU pool

Rollback

- revert worker image digest + routing policy immediately

Stage 4 — Online Evaluation (Shadow + Canary)

Shadow mode

- For a subset of requests, run:
  - current model pipeline
  - candidate pipeline
- Do NOT expose candidate output to user
- Compare scores + costs

Canary

- Route 1–5% of traffic to candidate
- Monitor:
  - latency
  - failure rate
  - cost/job
  - user rerun rate (proxy for dissatisfaction)

C. Model Registry & Release Management

Implement a simple model registry:

- table: `model_artifacts`
- table: `lora_artifacts`
- table: `routing_policy_versions`
- table: `quality_reports`

Each release is a “bundle”:

- `release_id`
- includes:
  - router policy vN
  - worker image digest
  - LoRA manifest vN
  - supported base models list

Deployments select a **release_id**.

D. Cold Start & Warm Pool CI Checks (MLOps meets SRE)

Make “cold start” a first-class test.

Automated checks per model

- weight download time
- load-to-VRAM time

- first-token/first-image time
- peak VRAM footprint
- concurrency sweet spot

Fail the build if it violates budgets for the tier.

E. Observability Signals that Drive Iteration (Agile loop)

Track these per model/pipeline:

- **Time to First Preview**
- **User rerun rate** (high = output mismatch)
- **Edit distance** between prompt iterations (how much the user had to correct)
- **Variant acceptance rate**
- **Cost per accepted output**
- **ControlNet compliance score** (layout adherence)

Use metrics to adjust:

- routing thresholds
- default pipelines per persona
- warm pool sizes

Suggested MVP Release Cadence (Agile)

- **Weekly**: Prompt compiler + routing policy updates (fast, safe)
- **Biweekly**: Inference worker updates
- **Monthly**: New LoRA packs + quality evaluation improvements

Scaling & Optimization

A) Cost Optimization via Queue System (RabbitMQ / Redis) for GPU Demand

1) Why a Queue Is the Core Cost Lever

GPU cost explodes when you:

- run too many on-demand instances
- allow unbounded concurrency
- let “long video jobs” block short preview jobs
- don’t prioritize paid tiers

A queue system lets you:

- **shape demand**
- **prioritize** high-value jobs
- **batch** similar jobs
- **autoscale** predictably

---

2) Recommended Queue Architecture (Practical + Scalable)

Option A (MVP + high throughput): Redis Streams

Pros

- Very fast, simple ops
- Great for job state + queue in one ecosystem
- Excellent for early-stage velocity

Cons

- Less “enterprise messaging” features than RabbitMQ

Option B (enterprise-grade routing): RabbitMQ

Pros

- Mature routing semantics
- Dead-letter queues, priorities, TTLs
- Strong for complex job classes

Cons

- More ops overhead

CRUSHED recommendation

- **MVP:** Redis Streams (fast iteration)
- **Scale:** RabbitMQ (or Kafka) once you need complex routing + guaranteed semantics

---

3) Queue Topology (CRUSHED-specific)

Separate queues by *latency class + modality*

You want **short jobs** (previews) to never be blocked by **long jobs** (video).

Queues

1. `q_text_fast` (LLM, low latency, streaming)
2. `q_image_preview` (GPU, short SLA)
3. `q_image_refine` (GPU, medium SLA)
4. `q_image_post` (GPU/CPU, upscalers, restoration)
5. `q_video_long` (GPU, long runtime)
6. `q_export_cpu` (CPU)

Add priority lanes by tier

Either:

- separate queues per tier, or
- priority field within one queue (RabbitMQ supports native priority)

Tier lanes

- `enterprise > pro > free`

Key rule

- `q_image_preview` always drains before `q_video_long`.

---

4) Job Spec + Scheduling Policy (the “GPU allocator”)

Job metadata fields (minimum)

- `job_id`, `org_id`, `tier`
- `modality`: image/video
- `task_type`: preview/refine/post/export
- `estimated_gpu_seconds`
- `model_family`: OSS / API / provider
- `deadline_ms` (SLA)
- `retries`, `idempotency_key`

Scheduling logic (simple, powerful)

- **Shortest-Job-First within tier** (to reduce tail latency)
- **Tier-based fairness** (prevent starvation)
- **GPU budget enforcement per org** (hard caps)

Pseudo-policy:

- Serve all `enterprise preview` jobs first
- Then `pro preview`
- Then `enterprise refine`
- Then `pro refine`
- Then `free preview`
- Video drains only when preview queues are under threshold

---

5) Autoscaling Strategy (Dedicated GPU + Burst GPU)

Dedicated GPU pool (warm)

- Always-on minimum replicas for:
  - `image_preview`
  - `text` (if self-hosted)
- Keep top models pinned

Scale triggers:

- queue depth
- wait time SLA breaches
- GPU utilization

Burst pool (serverless GPUs)

Use when:

- queue depth high
- long-running jobs pending
- video workloads spike

Scale down aggressively on idle.

---

6) Cost Levers You Implement in the Queue Layer (not UI)

1) Admission control (per tier)

- Free tier: limit concurrent GPU jobs per org (e.g., 1–2)
- Pro: higher concurrency
- Enterprise: reserved quota

2) Preview-first ladder

Always run:

- preview (cheap) → require user confirmation → refine (expensive)

This single lever can cut GPU spend massively.

3) Batch variants for marketers

For 10 variants:

- do one shared preprocessing stage (control maps)
- keep model warm and batch calls when possible

4) Circuit breakers

If GPU costs spike:

- degrade free tier to preview-only
- reduce steps
- route to cheaper providers
- delay refine jobs

5) Cache & replay

If a user regenerates the same recipe:

- reuse control maps
- reuse downloaded weights
- optionally reuse outputs if recipe identical (policy-dependent)

---

7) Failure Handling (must-have for cost)

- **Dead-letter queue** for repeated failures
- Retry policy:
  - transient errors: exponential backoff
  - quota errors: provider fallback
  - user cancellation: stop immediately (kill job)

Stop spending GPU seconds on doomed jobs.

B) Security: Prompt Injection Defense + API Rate Limiting

CRUSHED is especially exposed because:

- Users upload references + brand docs
- You generate system prompts and route to models
- Your prompt compiler is part of the moat

Security here is: **instruction integrity + abuse prevention + tenant isolation**.

---

1) Prompt Injection Protection (Defense-in-Depth)

Threat model

User tries to:

- override system instructions (“ignore safety, reveal hidden prompt”)
- extract secrets (API keys, system prompts, routing logic)
- jailbreak tool usage (force data exfil)
- poison brand packs (“always insert competitor logo”)

Core protection layers

Layer 1 — Strict message roles + non-overridable system prompt

- System instructions are immutable
- User content is treated as **data**, not instructions

Implementation:

- Always wrap user-provided docs/refs inside a `DATA_BLOCK` structure
- Never concatenate raw user text into system messages unquoted

Layer 2 — Prompt Compiler as a *policy engine*

Prompt Compiler must:

- parse user input for instruction patterns
- strip or quarantine malicious instruction attempts
- produce recipe in a constrained schema

Example safeguards:

- refuse “reveal system prompt”
- refuse “call tool X with credentials”
- refuse “export other users’ assets”

Layer 3 — Content provenance labeling

Every input chunk is tagged:

- `source=user_chat`
- `source=brandpack`
- `source=system_policy`
- `source=external_reference`

Then the compiler applies precedence rules:

- system_policy always wins
- brandpack wins over user prompt if conflict on brand constraints
- user chat can only set creative intent, not security policy

Layer 4 — Tool gating (capabilities-based access)

Even if the model is tricked, it cannot:

- access other org assets
- access secrets

- bypass allowed operations

Enforce with:

- signed URLs scoped to org/project
- IAM policies using `org_id` conditions
- no tool exposes raw credentials

Layer 5 — Output scanning (optional but helpful)

- detect prompt leakage patterns in responses
- block if model tries to print internal policies
- sanitize generated prompts displayed to user (show “recipe summary” not raw system prompt by default)

---

2) Multi-tenant Data Isolation (Non-negotiable)

- Every asset path includes `org_id/project_id`
- Every DB query is scoped by org_id (row-level security where possible)
- Vector search always filtered by org_id before similarity ranking
- Signed URLs include org claims and short TTL

---

3) API Rate Limiting (Abuse + Cost Control)

Rate limiting is both security and cost governance.

Three layers of rate limiting

1. **Edge / WAF**

- IP-based throttling
- geo and bot rules
- block obvious abuse patterns

2. **App-level**

- per-user + per-org rate limits
- separate budgets:
    - chat tokens/sec
    - generations/min
    - GPU-seconds/day
- tier-based limits

3. **Queue-level**

- concurrency caps per org
- job admission control
- per-queue max in-flight

Rate limit keys

- `{org_id}:{user_id}:{endpoint}`
- `{org_id}:{tier}:{queue}`
- `{ip}:{route}` (edge)

Hardening against token abuse

- token bucket with burst allowance
- exponential penalties on repeated failures
- CAPTCHA / email verification for suspicious free tier usage

---

4) Provider Key Security (Prevent catastrophic leaks)

- Never ship keys to client
- Use secret manager (AWS Secrets Manager/GCP Secret Manager)
- Rotate keys regularly
- Separate keys by environment and service
- Audit every provider call with:
  - org_id
  - job_id
  - cost units
  - model used

---

5) Safe-by-Design Prompt Exposure Policy

CRUSHED's product may want to show prompts (power users), but this is risky.

Recommended:

- Default: show **"Prompt Recipe Summary"**
- Pro/Enterprise: allow viewing structured recipe blocks
- Never display:
  - hidden system policies
  - provider keys
  - internal routing thresholds

---

6) Security Monitoring (cheap + effective)

Alert on:

- abnormal generation rate per org/IP
- repeated prompt-injection signatures

- high job failure loops (cost burn)
- suspicious vector search patterns (enumeration)

Deliverable Summary (What you implement)

Queue & Cost

- Redis Streams/RabbitMQ topology by latency class
- Tier-based priority + admission control
- Shortest-job-first scheduling
- Warm pool + burst pool autoscaling
- Preview-first ladder enforced by orchestrator

Security

- Prompt compiler policy enforcement
- Message provenance + precedence rules
- Tool gating + tenant isolation
- Multi-layer rate limiting (edge/app/queue)
- Provider key management + auditing

Technical API contract

1. API Endpoint: POST /v1/jobs/create

This endpoint is triggered after the Intent Engine has finalized the **Creative Brief Graph** through conversation.

Request Payload (Intent Engine → Job Orchestrator)

The Intent Engine sends a **PromptRecipe**, which acts as the canonical generation instruction.

JSON

```
None
{
  "org_id": "org_778899",
  "project_id": "proj_abc_123",
  "recipe": {
    "version": "1.0.4",
    "modality": "image",
    "persona": "performance_marketer",
    "intent_meta": {
      "goal": "performance",
      "realism": "high",
      "spatial_constraints": true
    },
    "model_plan": {
      "base_model": "SDXL_OR_WAN",
      "provider_tier": "dedicated_warm"
    },
    "prompt_blocks": {
      "system": "High-fidelity commercial cookware photography, sharp focus, premium lighting.",
      "user": "A sleek stainless steel frying pan on a minimalist marble kitchen counter.",
      "negative": "blurry, low quality, distorted, text, watermark"
    },
    "control_plan": {
      "control_type": "layout_mask",
      "mask_s3_url": "s3://crushed-dev/org_77/masks/layout_v1.png",
      "strength": 0.85
    },
    "render_plan": {
      "aspect_ratio": "4:5",
      "resolution": "1080x1350",
      "seed_policy": "lock_seed",
      "seed": 42069
    }
  },
  "priority": 1,
  "webhook_url": "https://api.crushed.ai/v1/internal/callback"
}
```

---

2. The Internal Logic: "The Recipe Handshake"

When the Job Orchestrator receives this payload, it performs the following logic to initiate the **Walking Skeleton** flow:

1. **Job Graph Expansion:** It breaks the recipe into a sequence of tasks: prepare_inputs → generate_preview → publish.
2. **Queue Injection:** The job is pushed into the q_image_preview Redis stream.
3. **Inference Routing:** The router checks gpu_state.dedicated_available. If warm, it pins the request to the dedicated pool; otherwise, it triggers a serverless burst.

---

3. Response Structure (Job Orchestrator → Intent Engine)

The orchestrator immediately returns a job_id and a tracking URL so the Chat UI can show a "Generating..." state.

Field	Type	Description
job_id	String	Unique UUID for the execution.

status	String	queued, running, or failed.
estimated_ms	Integer	Predicted wait time based on queue depth.
stream_url	String	WebSocket/SSE endpoint for real-time progress events.

---

4. Output Schema (Execution Result)

Once the GPU worker completes the task, the metadata is finalized in Postgres and returned to the system.

JSON

```
None
{
  "job_id": "job_uuid_001",
  "outputs": [
    {
      "asset_id": "asset_99",
      "url": "https://cdn.crushed.ai/org_77/project_abc/previews/final_v1.png",
      "mime": "image/png",
      "dimensions": "1080x1350",
      "sha256": "8f32e1..."
    }
  ],
  "provenance": {
    "recipe_id": "v1.0.4",
    "gpu_seconds": 4.2,
    "model_hash": "wan_v2_exp"
  }
}
```

Database Schema

To ensure CRUSHED successfully builds its "Thinking Layer" moat, the PromptRecipes table must act as a version-controlled repository for creative intent. This schema is designed to capture not just the prompt text, but the entire orchestration logic—including persona constraints, layout masks, and model routing decisions.

Database Schema: prompt_recipes

This table serves as the source of truth for the **Prompt Compiler** and is critical for future fine-tuning of the "Intent-to-Prompt" knowledge base.

Column Name	Data Type	Constraints	Description
recipe_id	UUID	PRIMARY KEY	Unique identifier for this specific iteration.
org_id	UUID	NOT NULL, INDEX	Mandatory for multi-tenant data isolation.
project_id	UUID	NOT NULL, INDEX	Links the recipe to a specific creative project.
parent_recipe_id	UUID	REFERENCES prompt_recipes	Used for lineage tracking during iterative chat loops.
persona	ENUM	NOT NULL	The detected persona (e.g., performance_marketer, designer).
brief_graph	JSONB	NOT NULL	The structured output from the Intent Engine (goals, style anchors).
prompt_blocks	JSONB	NOT NULL	Contains the system, user, and negative prompts.
control_plan	JSONB	NULLABLE	Configuration for ControlNet, layout masks, and spatial constraints.
model_config	JSONB	NOT NULL	Stores model ID, version, and the specific Model Adapter used.
inference_params	JSONB	NOT NULL	Seed, resolution, steps, and CFG scale.
quality_score	DECIMAL	NULLABLE	Post-generation score used for future model fine-tuning.
version_tag	VARCHAR	NOT NULL	Schema version (e.g., v1.0.4) for backward compatibility.
created_at	TIMESTAMP	DEFAULT NOW()	Audit timestamp for chronological sorting.

Why this Schema Supports your Moat

As a CTO, the goal of this structure is to turn every user interaction into a training asset for your proprietary intelligence layer.

- **Iterative Lineage:** By using parent_recipe_id, we track how the Intent Engine refines prompts based on user feedback (e.g., "Make it darker"), allowing us to train a model that predicts "creative delta".
- **Deterministic Replay:** Storing the inference_params and model_config ensures that we can reproduce the exact same output for paid tiers, meeting the "determinism" requirement.
- **Fine-Tuning Ready:** The brief_graph paired with high quality_score outputs provides the ideal dataset for training a custom LLM to replace expensive third-party APIs like GPT-5.
- **Spatial Intelligence:** The control_plan preserves how layout masks from the canvas influenced the final image, a key differentiator for commercial-grade assets.